Лабораторная работа №3 по курсу дискретного анализа: исследование качества программы

Выполнил студент группы 08-215 МАИ *Матвеев Семен*.

Цель работы

Целью работы является исследование скорости выполнения и потребления динамической памяти программы-словаря на красно-чёрном дереве из файла `lab2.cpp`. Для профилирования использовались утилита `gprof` и библиотека `dmalloc`.

Методика

Для эксперимента был подготовлен синтетический входной файл на 226669 команд. В нём выполнялись:

- добавление 80000 ключей;
- поиск всех добавленных ключей;
- сохранение дерева в бинарный файл;
- удаление каждого третьего ключа;
- загрузка ранее сохранённого дерева;
- поиск каждого второго ключа.

Ключи генерировались детерминированно из строчных латинских букв, чтобы вход соответствовал условию задачи. Вывод программы перенаправлялся в файл, так как печать ответов на экран не относится к работе структуры данных и может исказить измерения.

Использованные команды

```
g++ -std=c++17 -O0 -pg -g -Wall -Wextra -no-pie lab2.cpp -o lab2_gprof_o0
./lab2_gprof_o0 < /tmp/lab2_profile_input.txt > /tmp/lab2_profile_output_o0.txt
gprof ./lab2_gprof_o0 gmon.out > /tmp/lab2_gprof_o0_report.txt

g++ -std=c++17 -O0 -g -Wall -Wextra -DDMALLOC -DDMALLOC_FUNC_CHECK \
    lab2.cpp -o lab2_dmalloc_fixed -ldmallocthcxx -ldmallocth
eval "$(dmalloc -b -l /tmp/lab2_dmalloc_fixed.log -i 1000 high)"
./lab2_dmalloc_fixed < /tmp/lab2_dmalloc_input.txt > /tmp/lab2_dmalloc_fixed_output.txt
```

Дневник выполнения работы

1. Проверено наличие инструментов: `gprof` и `dmalloc` установлены в системе.
2. Собрана диагностическая версия программы с флагами `-pg -g`. Первый профиль с оптимизацией `-O2` оказался менее удобен для анализа, так как часть функций была заинлайнена. Поэтому основной отчёт `gprof` снимался для сборки `-O0`.

3. Сгенерирован большой тестовый вход и выполнен прогон через `gprof`. По результатам выделены основные горячие места: сравнение строк, рекурсивный поиск, операции вставки и сериализация дерева.

4. Собрана версия с `dmalloc`. При попытке подключить `dmalloc.h` через ключ компилятора `-include` заголовок попал раньше стандартных C/C++ заголовков и вызвал конфликт макросов. Ошибка устранена явным условным включением `dmalloc.h` после стандартных заголовков.

5. Для поддержки `DMALLOC_FUNC_CHECK` вызов `std::memcpy` был заменён на `::memcpy`, так как `dmalloc` подменяет имя `memcpy` макросом.

6. По результатам профилирования исправлен неэффективный участок сериализации: вместо создания временной строки для каждой вершины данные вершины теперь сразу дописываются в общий буфер.

7. Дополнительно исправлено копирование пустого `TVector`: теперь при размере 0 не выполняется `new T[0]`. Это убрало предупреждение компилятора и сделало управление памятью аккуратнее.

8. После исправлений программа была пересобрана, повторно проверена через `gprof` и `dmalloc`. Вывод исправленной версии совпал с выводом предыдущей версии на том же входе.

Результаты gprof до исправлений

Основные строки плоского профиля для исходной версии:

Функция	**Доля времени**	**Число вызовов**
`TVector<char>::operator<`	13.05%	4639369
`TVector<char>::operator==`	8.70%	3492369
`TVector<char>::reserve`	8.70%	1306767
`TVector<char>::append(const char*, size_t)`	8.70%	320000
`RedBlackTree::RecursiveSearchNode`	8.70%	226667
`RedBlackTree::WriteTreeToString`	19.0% в call graph	1 + 89694 рекурсивных

Профиль показывает, что основная стоимость ожидаемо находится в сравнении ключей при поиске в дереве. Для операций `Save` заметным оказался вспомогательный код сериализации: на каждую вершину создавался временный `TString`, который затем копировался в общий буфер.

Результаты dmalloc до исправлений

На тесте из 10000 ключей `dmalloc` показал:

- `new`: 20001 вызов, `delete`: 20001 вызов;
- `malloc`: 199853 вызова, `free`: 199846 вызовов;

- всего выделено: 5359026 байт;
- максимальное использование памяти: 2067560 байт.

Количество `new`/`delete` совпало, поэтому утечек узлов дерева и массивов `TVector`, выделенных пользовательским кодом, обнаружено не было. Оставшиеся блоки `malloc` в отчёте помечены как `unknown`; они относятся к окружению выполнения и стандартным потокам, так как `dmalloc_shutdown` вызывается до полного разрушения статических объектов стандартной библиотеки.

Найденные недочёты и исправления

1. В сериализации дерева функция `GetNodeString` создавала отдельный временный буфер для каждой вершины. Это приводило к лишним выделениям памяти и копированиям. Исправление: добавлена функция `AppendNodeBytes`, которая сразу записывает байты вершины в общий буфер.
2. В копирующем конструкторе и операторе присваивания `TVector` выполнялось выделение `new T[0]` для пустого вектора. Формально это допустимо, но компилятор выдавал предупреждение, а сама операция не нужна. Исправление: память выделяется только при размере больше нуля.
3. Для корректной работы `dmalloc` с программой на C++ добавлены условная инициализация и завершение библиотеки. Это не влияет на обычную сборку программы, так как код активен только при `-DDMALLOC`.

Сравнение после исправлений

Показатель	**До**	**После**
`TVector<char>::append(const char*, size_t)`, вызовы	320000	240000
`TVector<char>::TVector()`, вызовы	840009	760009
`TVector<char>::~TVector()`, вызовы	1226678	1146678
`WriteTreeToString`, суммарно по call graph	43.63 мс	21.91 мс
`dmalloc`: всего выделено	5359026 байт	4800697 байт
`dmalloc`: `malloc`	199853	159860
`dmalloc`: `new/delete`	20001/20001	20001/20001

Исправление не изменило формат вывода программы: результат исправленной версии совпал с результатом исходной версии на одинаковом входном файле. Основное улучшение проявилось в операции сохранения: исчезло 80000 временных строк, количество вызовов `append` уменьшилось на 25%, а суммарное время сериализации в профиле снизилось примерно в 2 раза.

Выводы

Профилирование подтвердило, что программа в основном тратит время на сравнение ключей при поиске, вставке и удалении в красно-чёрном дереве. Это соответствует выбранной структуре данных: каждая операция выполняет последовательность сравнений вдоль пути от корня к вершине.

Проверка `dmalloc` не выявила утечек в пользовательских структурах данных: количество вызовов `new` и `delete` совпадает. Найденный по `gprof` недочёт в сериализации был исправлен, после чего уменьшилось число выделений памяти и копирований при сохранении дерева. В результате программа сохранила прежнее поведение, но стала аккуратнее по работе с динамической памятью и быстрее в операции `Save`.